

**NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS**

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

**Software Project Report on the Topic:
Development of an ML Model for Violation Classification
from Photos**

Fulfilled by:

Student of the Group БПАД232
Rodionova Anna Mikhailovna

Assessed by the Project Supervisor:

Smirnov Nikita Sergeevich
Faculty of Computer Science, HSE University

Moscow, 25 May 2026

Contents

Abstract	3
1 Introduction	4
1.1 Goals and contributions	4
2 Overview of bibliographical sources	5
2.1 Traditional approaches to image classification	5
2.2 Deep learning in infrastructure monitoring	5
2.3 Image similarity and verification	6
2.4 Image quality assessment in computer vision	6
2.5 Gap analysis	6
3 Problem formulation and system requirements	7
3.1 Formal problem statement	7
3.2 Functional requirements	7
3.3 Technical constraints	8
4 Materials and methods	8
4.1 Description of the source data	8
4.2 Manual data labeling	9
4.3 Automated data cleaning	10
4.4 Sampling strategy for training	11
4.5 Pipeline architecture	12
4.5.1 Module 1: category router	13
4.5.2 Module 2: location verification	13
4.5.3 Module 3: repair quality assessment	13
4.5.4 Module 4: decision router	13
4.6 Siamese network based on ResNet50	14
4.7 Synthetic-negative approach for the quality-assessment module	14
5 Experimental setup	15
5.1 Hardware and software	15
5.2 Training hyperparameters	15
5.3 Data augmentations	16
5.4 Pipeline evaluation protocol	16
5.5 Quality metrics	17
6 Results	18
6.1 Module 2: training the Location Verification models	18
6.2 Module 3: training the Quality Assessment models	20
6.3 End-to-end pipeline evaluation	21

6.4	Generalization check: SEEN vs UNSEEN	23
6.5	Threshold sensitivity	24
6.6	Ablation: pipeline without Module 3	25
7	Discussion	26
7.1	Why the unified model outperformed the specialized one	26
7.2	Limitations of the synthetic-negatives approach	26
7.3	Other limitations	27
7.4	Operational implications	28
8	Conclusion	29
	References	30
	Appendix A. Project repository structure	31

Abstract

This work develops a system for automatic verification of urban infrastructure repair tasks based on before/after photographs. Moscow city services produce approximately 196,858 such pairs per year across three defect categories (damaged asphalt pavement, untended lawns, broken curb stones); in the current workflow each pair requires manual visual review by an inspector. Even partial automation, in which the system handles clear-cut cases and forwards only ambiguous pairs to humans, yields labor savings.

We design and implement a four-module pipeline: (i) a category router that determines the violation type from the task description; (ii) a Siamese ResNet50 network that verifies whether both photographs were taken at the same physical location; (iii) a second Siamese ResNet50 that estimates whether the violation was actually repaired; (iv) a rule-based decision router that produces an ACCEPT, REJECT, or HUMAN_REVIEW label. We additionally run a comparative experiment between two design philosophies for the location-verification module: three category-specialized models (one per category) versus a single unified model trained on the mixed corpus.

On a held-out manually annotated test set of 6,567 pairs not used in training, the unified model outperforms the specialized ensemble on both key metrics: auto-accuracy is 97.35% vs 96.66%, and the critical false-positive rate is 2.13% vs 19.15% (a roughly ninefold reduction). The initial hypothesis favoring specialization is rejected: a larger training corpus turned out to be more important than narrow specialization. An additional ablation experiment shows that removing the quality-assessment module from the pipeline does not degrade any key metric. The synthetic-negatives approach in its current formulation is ineffective, and a simpler architecture is justified for the production version. The final product is implemented as a command-line utility that consumes Excel files in the format used by city services and produces an Excel file with decisions.

Keywords

Urban infrastructure monitoring; computer vision; deep learning; Siamese neural networks; ResNet50; image-based location verification; repair verification; image quality assessment; transfer learning; confidence-threshold filtering; batch processing.

1 Introduction

Moscow municipal inspection services routinely record violations of urban infrastructure: damaged road surface, broken curb stones, untended or uncleaned lawns. Each such violation is forwarded to the responsible contractor or district administration, who, after repair, submits a photograph of the completed work. The inspector must then confirm two independent facts: first, that the repair photograph was taken at the same location as the original violation photograph; second, that the quality of the repair meets the regulations.

A large-scale system processing tens of thousands of cases per day makes manual review labor-intensive and potentially costly. At the same time, years of operation have accumulated an archive of before/after pairs together with expert decisions made by inspectors, which makes supervised machine-learning approaches applicable. The present work transforms this archive into an automated decision-support tool that handles routine cases without human involvement and isolates ambiguous pairs for manual review.

The underlying computer-vision problem is non-trivial. The two photographs are usually taken by different people at different times (days, weeks or months apart), using different cameras under varying lighting and weather conditions. The violation itself often occupies only a small fraction of the frame (for example, a pothole a few centimeters across in a wide street view). A portion of the submitted “photographs” are in fact PDF documents, GIS-system screenshots, or other irrelevant files mistakenly uploaded as evidence. An effective model must simultaneously (a) tolerate substantial changes in lighting, viewing angle and capture device, (b) confidently recognize semantic differences between distinct locations, and (c) flag obviously invalid submissions.

1.1 Goals and contributions

The main goals are:

- to design an end-to-end pipeline capable of accepting Excel files with tasks in the current format used by city services and producing one of three decisions per task: ACCEPT, REJECT, or HUMAN_REVIEW;
- to compare two architectural approaches for the central image-similarity module: an ensemble of three category-specialized models versus a single unified model trained on the mixed corpus;
- to investigate whether a separate quality-assessment module trained with synthetic negative examples can effectively complement the location-verification module.

The contributions are:

- manual labeling of a subset of 8 000 pairs with two classes (good / bad) using

- a purpose-built graphical interface; the resulting `pair_decisions.csv` file is used as an independent test set throughout subsequent stages;
- trained Siamese networks for location verification with validation accuracy up to 96.81% and AUC of 0.9942; the unified-model variant covers all three categories (“asphalt”, “lawn”, “curb”) with a single architecture;
 - a comparative study of specialized vs unified approaches showing that the unified model delivers approximately a ninefold reduction in critical false-positive rate (from 19.15% to 2.13%) on the held-out test set;
 - a fully functional command-line utility (`mvp_pipeline.py`) that implements the pipeline from Excel input to Excel output and is suitable for integration into the existing city-services workflow.

2 Overview of bibliographical sources

2.1 Traditional approaches to image classification

Early computer-vision systems for classification tasks relied on hand-crafted visual features (texture descriptors, Histograms of Oriented Gradients (HOG), Gabor filters) combined with classical classifiers such as support vector machines (SVM) or random forests [1, 2]. Oliveira and Correia [2] reached 95% accuracy in detecting road-surface cracks using morphological features; however, their proposed system required controlled laboratory imaging conditions. These systems generalize poorly to the kinds of images typical of municipal inspections, where lighting and viewing angle vary widely. They work only under standardized conditions.

2.2 Deep learning in infrastructure monitoring

Convolutional neural networks (CNNs) and transfer learning brought a qualitative shift to detection and monitoring. Gopalakrishnan et al. [3] showed that fine-tuning ImageNet-pretrained models achieves above 90% accuracy in pavement-defect classification. The ResNet architecture introduced by He et al. [4], with its residual connections, made it possible to train deeper models and improved the ability of networks to extract visual features of complex textures (in particular, defective surfaces). The EfficientNet family proposed by Tan and Le [5] advanced the accuracy-cost trade-off through compound scaling, that is, coordinated scaling of depth, width and resolution.

Such pretrained backbones admit fine-tuning to a specific downstream task with substantial savings in data and compute. However, the vast majority of existing work focuses on single-object detection (binary “defect / no

defect” classification) on a single image, whereas verifying repair of a violation requires pairwise comparison of two images and a joint assessment of their correspondence.

2.3 Image similarity and verification

Verifying location consistency from two photographs reduces to image-similarity search or visual place recognition. Bromley et al. [6] proposed Siamese neural networks to learn a feature-space embedding in which images of the same object lie closer than images of different objects. Schroff et al. [7] extended this approach in the FaceNet system, applying a triplet loss for face verification.

Perceptual hashing [8] is a simpler baseline. It offers fast approximate comparison but is sensitive to substantial changes in angle and lighting, and is inferior in this respect to embedding-based methods built on deep features. Metric-learning methods are more appropriate for the present task, since they handle the variability typical of field photography.

2.4 Image quality assessment in computer vision

Image quality can be assessed in two ways: as direct classification (acceptable / not acceptable), or by comparing the “before” and “after” images. Wang et al. [9] proposed the SSIM (Structural Similarity Index) metric, which quantifies structural similarity and is widely used for image-quality assessment.

Zhang et al. [10] showed that the LPIPS (Learned Perceptual Image Patch Similarity) metric, based on distances in the feature space of a trained neural network, aligns better with human perception than pixel-wise metrics MSE and PSNR. Combining the classification approach (is the repair acceptable?) with the comparative perceptual approach provides complementary information for more reliable decisions.

2.5 Gap analysis

The review above reveals several gaps that the present work addresses:

- task mismatch: most published work focuses on single-image defect detection [1–3], whereas verifying repair requires comparison of an image pair;
- anti-fraud: location verification as a means of preventing photo substitution is rarely treated as a key requirement of existing systems;
- operational integration: confidence-threshold filtering for human-in-the-loop processes is barely discussed in the academic literature, even though it is essential for real-world operation;

- domain specifics: existing detection systems are oriented primarily towards public roads and highways [2, 3], not towards municipal violations in the urban environment (sidewalks, lawns, curb stones).

The present work closes these gaps by building an end-to-end pipeline with simultaneous location verification and quality assessment, and with a tunable threshold-based decision system that meets the real operational requirements of a municipal system.

3 Problem formulation and system requirements

3.1 Formal problem statement

Inputs. A pair of images (b, a) , where b is the violation photograph (“before”) and a is the repair photograph (“after”), together with a category label $c \in \{\text{asphalt, lawn, curb}\}$ determined from the textual description of the task.

Outputs. A discrete decision

$$d \in \{\text{ACCEPT, REJECT, HUMAN_REVIEW}\}$$

and a pair of real-valued scores $(p_{\text{loc}}, p_{\text{qa}}) \in [0, 1]^2$, where p_{loc} is the estimated probability that both photographs were taken at the same location and p_{qa} is the estimated probability that the repair was actually performed.

Decision rule. The output decision d is determined from the pair $(p_{\text{loc}}, p_{\text{qa}})$ by the following rules:

- if $p_{\text{loc}} < \tau_{\text{reject}}^{\text{loc}}$, then $d = \text{REJECT}$ (different locations);
- if $p_{\text{qa}} < \tau_{\text{reject}}^{\text{qa}}$, then $d = \text{REJECT}$ (a quality issue is detected);
- if $p_{\text{loc}} > \tau_{\text{accept}}^{\text{loc}}$ and $p_{\text{qa}} > \tau_{\text{accept}}^{\text{qa}}$, then $d = \text{ACCEPT}$ (high confidence);
- otherwise $d = \text{HUMAN_REVIEW}$ (forwarded to an inspector).

The thresholds $(\tau_{\text{accept}}^{\text{loc}}, \tau_{\text{accept}}^{\text{qa}}, \tau_{\text{reject}}^{\text{loc}}, \tau_{\text{reject}}^{\text{qa}})$ are set to (0.95, 0.80, 0.30, 0.30) to keep the system conservative. Under any uncertainty on either axis, the pipeline forwards the task to an inspector rather than making an automatic decision.

3.2 Functional requirements

- **FR1.** Violation-type coverage: at least three categories (asphalt, lawn, curb).
- **FR2.** Location verification: reliable detection of location-mismatch cases under the variability of field photographs.

- **FR3.** Repair quality assessment: minimization of false-positive decisions (where the system wrongly approves work that was not performed) as the most important operational metric.
- **FR4.** Confidence calibration: outputs provided as probability scores with tunable thresholds (e.g. in the range 0.85–0.99).
- **FR5.** MVP batch-processing mode: reading an Excel file with image URLs and producing an output Excel file with predictions and automatic decisions.
- **FR6.** Throughput: efficient processing of large batches with empirically verified throughput.

3.3 Technical constraints

- **TC1.** Data protection: inference must be performed on the customer’s own infrastructure without calling external cloud APIs.
- **TC2.** Interpretability: for HUMAN_REVIEW cases, visual explanations are desirable (for example, Grad-CAM-style heat maps).
- **TC3.** Robustness: correct handling of common image formats and degraded content; stable behavior under failures.
- **TC4.** Scalability: ability to add new violation categories and incrementally update models as new labeled data accumulates.

4 Materials and methods

4.1 Description of the source data

City services provided three Excel files, one per violation category. Each row of a file corresponds to one verification task and contains a free-text task title (e.g. “Potholes and rutting on the road surface”), the URL of the “before” photograph (the recorded violation), and the URL of the “after” photograph (the claimed repair). Final per-category data volumes are summarized in Table 1. The exact time span of data collection is not explicitly stated in the source files; for this work, the 196 858 pairs are treated as the available archive, which city services characterize as the annual flow of tasks across these three categories. This figure is used below as a reference value for throughput estimation (FR6) and inspector workload.

Table 1. Composition of the source dataset by category.

Category	Keywords in task name	Pairs
Damaged asphalt pavement	ямы, выбоины	42 139
Untended lawn	газон, убранный	55 397
Broken curb stones	бортовой камень	99 322
Total		196 858

The source data contain only records of accepted tasks, i.e. tasks for which the contractor has already been paid. In other words, the data do not provide explicit “correctly / incorrectly repaired” labels. This made it necessary to perform an independent manual labeling of our own, described below in Section 4.2, for the purpose of independent quality evaluation.

4.2 Manual data labeling

To build an independent reference test set, a custom Tkinter-based graphical interface (script `filter_pairs.py`) was developed that displays the two photographs side by side and asks the annotator to assign one of three labels:

- **good**: the pair is valid, both photographs are normal images of urban environment, and the violation has been repaired;
- **bad**: at least one of the photographs is invalid (a PDF document, a GIS-system screenshot, an empty picture, an obviously different location, etc.);
- **unsure**: the annotator is uncertain.

The author labeled 8 000 randomly selected pairs, stratified by category. After excluding 0 **unsure** labels, 7 888 **good** and 112 **bad** pairs remain (1.4% of all labeled pairs). The resulting file `pair_decisions.csv` is used in Section 6 as an independent test set for the final pipeline evaluation.

A low fraction of invalid pairs (1.4%) might at first glance suggest a trivial task. However, in terms of operational significance, these 112 pairs are the most valuable: every wrongly accepted bad pair means payment for work that was not actually performed. A balanced quality evaluation must therefore pay special attention to the critical false-positive rate (see 5.5).

Labeling was performed by a single annotator, which precludes computing inter-annotator agreement. For the bad category, with its low frequency, this is especially important, since the determination “the violation is clearly not repaired” may rely on subjective judgments in ambiguous cases. Section 7.2 discusses this limitation in more detail.

4.3 Automated data cleaning

Three automated cleaning stages were applied to the full set of 196 858 pairs, supplemented by a single post-processing step that resolves labeling inconsistencies.

Stage 1. URL extension filtering. A portion of the URLs point to files that are not images (.pdf, .doc, .heif, etc.), which the loader cannot process. The corresponding rows are excluded (scripts `filter_pairs.py` and `prepare_training_data.py`).

Stage 2. Anomaly detection using CLIP [12]. Each “after” photograph is scored against 17 text prompts grouped into four categories: `real_photo` (a normal photograph of urban environment), `ui_screenshot` (a screenshot of a GIS, map or administrative system), `document_photo` (a photograph of a paper document or report), `signage_photo` (a close-up of a plate or road sign). Similarities are averaged within each category, and the pair is assigned to the category with the highest mean. Pairs assigned to anything other than `real_photo` are flagged for renewed manual review. The implementation uses the OpenAI CLIP ViT-B/32 model in the `detect_anomalies.py` script.

Stage 3. Manual re-review of suspicious pairs. Pairs flagged at stage 2 are inspected in a second graphical interface (`review_suspicious.py`) and reclassified as good/bad based on visual analysis.

Stage 4. Post-processing of labeling conflicts. Based on the results of the initial labeling and the automatic verification, the `fix_bad_decisions.py` script automatically identifies conflicting cases (e.g. when the same photograph appears in several pairs with incompatible labels) and applies reconciliation rules.

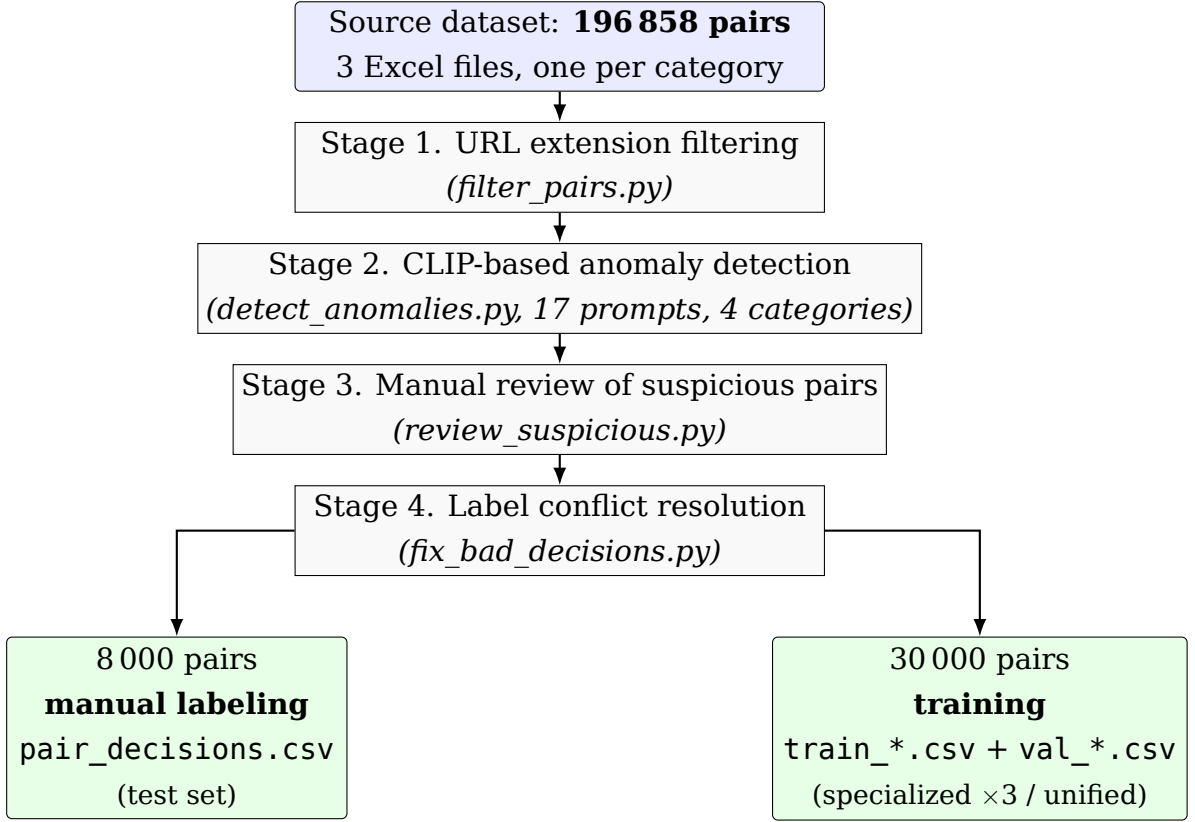


Figure 1. Data preprocessing pipeline: from 196 858 raw pairs to final training and test sets.

4.4 Sampling strategy for training

Training on the full set of 196 858 pairs would require about 60 GB of disk space for image storage, which exceeds the available budget on both the personal training machine (RTX 3070, 8 GB of VRAM) and Kaggle (20 GB of disk per session). Two training-set preparation modes were therefore considered.

The specialized models were trained on a stratified random sample of 10 000 pairs per category (asphalt, lawn, curb), with an 80/20 train/validation split. Within each subsample, the within-category shuffle strategy was used to produce negatives: “after” photos were shuffled across tasks within the same category. This produced a balanced binary classification task in which label 1 corresponds to a genuine pair of the same location, and label 0 corresponds to a shuffled pair of different locations.

The unified model was trained on a stratified random sample of 30 000 pairs (10 000 from each category, merged into a single pool), with the same 80/20 split and the same negative-generation scheme (within-category shuffle). The 30 000-pair volume corresponds to approximately 15.2% of the full source set, which provides sufficient coverage of the data variety while keeping disk and compute requirements moderate.

All stochastic steps of data preparation (random pair selection, shuffling

for negative generation, train/val split stratified by “category, label” pairs) use a fixed seed of 42, which guarantees reproducibility of the composition of training and validation sets across runs. The neural-network training procedure itself (the backbone is initialized via ImageNet-pretrained weights, so only the head undergoes random initialization, and batch ordering inside the DataLoader varies between runs) does not use a fixed seed: this is a methodological oversight whose consequences are discussed in 7.2.

4.5 Pipeline architecture

The system is structured as four sequential modules (see Fig. 2). Each task from the input Excel file passes sequentially through all the modules, after which the pipeline emits a single decision. Modules 2 and 3 are trained neural networks, while Modules 1 and 4 are implemented as deterministic rules.

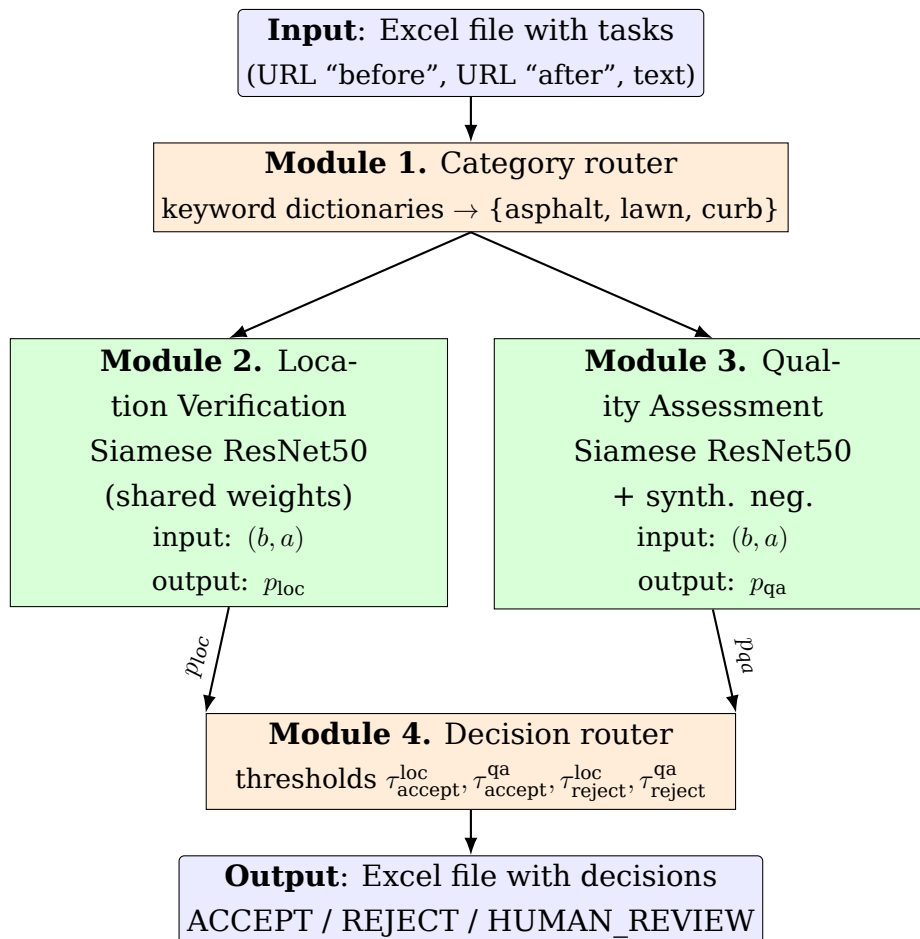


Figure 2. Pipeline architecture for violation-repair verification. Modules 2 and 3 are trained Siamese networks (green boxes); Modules 1 and 4 are deterministic rules (orange boxes).

4.5.1 Module 1: category router

The module reads the free-text task title from the column “Task title” and matches it against pre-defined keyword dictionaries:

- asphalt: ['асфальт', 'выбоин', 'ям', 'покрытии'];
- lawn: ['газон', 'убранный'];
- curb: ['бортов', 'камн'].

The determined category is used at the subsequent stages to select the corresponding model in the specialized regime, as well as for statistics and logging. On the labeled test set of 8 000 pairs, the router successfully covered every task: none was assigned to the pseudo-category unknown. The character of this statement should be qualified: the reference category for each pair is determined by the source Excel file from which the pair originated, not by semantic analysis of the task. What is tested is not “does the module understand the task” but rather *coverage* of the city services’ current title format by the keyword dictionaries used. On the source format studied, this coverage is 100%, and a more sophisticated classification model is not required for this module. Should the title format change, the keyword dictionaries would need revision, or replacement by a more reliable classifier.

4.5.2 Module 2: location verification

A Siamese network based on ResNet50 (Location Verification) classifies whether both photographs (“before” and “after”) depict the same physical location. A detailed description of the architecture, training loss, and training procedure is given in Section 4.6.

4.5.3 Module 3: repair quality assessment

A second Siamese network (Quality Assessment), trained using the synthetic-negatives strategy (Section 4.7), outputs the probability that the “after” photograph actually depicts the same location with the violation repaired, as opposed to, for example, the same location re-photographed without any repair work having been done.

4.5.4 Module 4: decision router

The module (Decision Router) takes as input the probabilities p_{loc} and p_{qa} produced by Modules 2 and 3 and produces the final decision following the rules introduced in Section 3.1. All threshold values are utility parameters and can be overridden via command-line flags.

4.6 Siamese network based on ResNet50

The central architecture is a Siamese network with two identical, weight-sharing branches. Each branch uses a ResNet50 backbone (He et al. [4]) pretrained on ImageNet (IMAGENET1K_V2 weights from torchvision). The final fully connected classification layer of ResNet50 is replaced with an identity mapping (`nn.Identity`), so that each input image of size 224×224 pixels is transformed into a feature vector of dimension 2048.

The resulting feature vectors f_1, f_2 are combined via a “similarity head” that takes as input the concatenation $[f_1; f_2; |f_1 - f_2|]$. This is a standard scheme going back to Reimers and Gurevych [11], which gives the head direct access to both the representations themselves and their element-wise difference. The head is implemented as a small multi-layer perceptron:

`Linear(2048×3 → 512) → ReLU → Dropout( $p = 0.5$ ) → Linear(512 → 1) → Sigmoid.`

The output scalar value is interpreted as the probability that the pair belongs to the “same location” class (label = 1). Training is performed with binary cross-entropy loss (`BCEWithLogitsLoss`). The total number of trainable parameters is approximately 26.7 million (including the backbone and head).

4.7 Synthetic-negative approach for the quality-assessment module

The principal difficulty in training Module 3 is that the source data has no explicit examples of the class “violation not repaired”: by construction, all tasks in the source files have already been accepted by the customer, and the “after” photographs reflect the fact of work being performed. To train the quality-assessment classifier nevertheless, a synthetic-negatives strategy was applied: for every genuine pair (b, a) two training examples are formed:

- positive (label = 1): the pair (b, a) itself, i.e. an actual repair;
- synthetic negative (label = 0, synthetic = 1): the pair (b, b') , where b' denotes a copy of b processed with a strong set of augmentations intended to mimic the visual signature of the same location re-photographed later without any repair work having been done.

The augmentation used for synthetic negatives models the following sources of variability of real field re-photography:

- perspective and framing changes: `RandomResizedCrop` with scale range $[0.65, 0.95]$, `RandomPerspective` with distortion 0.25, `RandomAffine` with translate up to 8% and shear up to 8° ;
- rotation: `RandomRotation` up to $\pm 12^\circ$;

- time of day: RandomGamma in the range [0.5, 1.7], modelling dawn, noon, sunset and overcast weather;
- color temperature: a custom RandomColorTemperature, performing opposite shifts of the red and blue channels and modelling warm (sunny) and cold (overcast) lighting;
- camera-characteristic changes: RandomAdjustSharpness, RandomAutocontrast, GaussianBlur.

Geometric augmentation artifacts can leak: directly applying a rotation or affine shift to a fixed-size image produces black borders, which the model could trivially use as a “synthetic example” indicator. To avoid this, we implemented a custom SafeGeometricAug class that performs all geometric transformations on an enlarged copy of the image with reflect padding at the boundaries, after which the final 224×224 image is cropped from the center. With this approach, black borders never appear in the output frame at any rotation angle.

Limitations of the proposed approach are discussed in Section 7.2.

5 Experimental setup

5.1 Hardware and software

All location-verification models (specialized) and all quality-assessment models were trained on a personal workstation with the following specifications: NVIDIA RTX 3070 GPU (8 GB VRAM, compute capability 8.6), Intel Core i5-13600K CPU, 32 GB of RAM. The unified location-verification model was trained in the free Kaggle GPU environment (NVIDIA Tesla T4, 16 GB VRAM) due to the larger training-set volume and disk-space requirements.

Software stack: Python 3.10, PyTorch 2.x, torchvision (with ImageNet1K V2 weights for ResNet50), scikit-learn, OpenCV, pandas, Pillow, requests; the anomaly-detection stage used the OpenAI CLIP ViT-B/32 model [12]; matplotlib was used for visualization; the docx (Node.js) package was used to generate the final report.

5.2 Training hyperparameters

All trained models use a single set of hyperparameters, listed in Table 2.

Table 2. Training hyperparameters used for all ResNet50-based Siamese models.

Hyperparameter	Value
Backbone architecture	ResNet50, pretrained on ImageNet-1K V2
Input image size	224×224
Batch size	32 (specialized), 64 (unified)
Optimizer	AdamW
Initial learning rate	1×10^{-4}
Weight decay	1×10^{-4}
LR schedule	Cosine annealing, 10-epoch cycle
Head dropout	0.5
Loss function	Binary cross-entropy (BCEWithLogitsLoss)
Maximum epochs	10
Early stopping patience	3 epochs without <code>val_acc</code> improvement

5.3 Data augmentations

For all positive pairs (label = 1), a single standard set of augmentations is applied during training. These augmentations simulate the normal variability of field photography and act as a regularizer:

- random resize and crop to 224×224 (RandomResize + RandomCrop);
- horizontal flip with probability 0.5 (RandomHorizontalFlip);
- random color jitter (ColorJitter with brightness = 0.25, contrast = 0.25, saturation = 0.20, hue = 0.05);
- random gamma correction (RandomGamma in the range [0.75, 1.35]).

For synthetic negative pairs (label = 0, synthetic = 1), when training the quality-assessment module, the strengthened augmentation set described in Section 4.7 is applied. At validation time, all augmentations are applied with a deterministic seed (seed depends on the sample index) to ensure reproducibility of validation metrics across runs.

5.4 Pipeline evaluation protocol

A fair evaluation on the test set requires that pairs from the labeled file `pair_decisions.csv` not appear in training. Because the training and validation files of each category contain paths to cached images, the following overlap-detection procedure was used.

First, the set of basenames (MD5 hashes of URLs) of all photographs used in any training or validation split is collected. Then the 8 000 labeled

pairs are partitioned into two subsets:

- **SEEN**: pairs for which at least one of the two photographs appeared in some training or validation file. This subset is used as a sanity check, since metrics on it should be slightly higher than on UNSEEN: the model has already seen part of its inputs;
- **UNSEEN**: pairs for which neither photograph appears in any training set. This is the true held-out test set and the main source of metrics reported in Section 6.

Of the 8 000 labeled pairs, 23 could not be evaluated because the corresponding image URLs are no longer reachable. Thus, of the 8 000 labeled pairs, 7 977 valid predictions are available at evaluation time, distributed as 1 410 in SEEN and 6 567 in UNSEEN. In terms of test-set structure before checking image availability, the total number of UNSEEN pairs is 6 590 (including the 23 with no prediction). All 23 lost pairs fell into UNSEEN, with zero losses in SEEN: this is a natural consequence of the caching mechanism. Photographs from training and validation sets were preloaded into the local cache during data preparation and training, so their availability is guaranteed. For UNSEEN pairs, which do not overlap with training, the photographs had to be downloaded again at evaluation time, and a portion of the original city-services URLs had become unresponsive by then. The metrics below are computed only on the 6 567 valid-prediction pairs, while the SEEN/UNSEEN ratio is compared to the theoretically expected overlap of the test and training samples by total counts (1 410 and 6 590; see Section 6).

5.5 Quality metrics

On the held-out UNSEEN set, four pipeline-quality metrics are computed and reported:

- **Auto-accuracy**: the fraction of correct decisions among tasks for which the pipeline made an automatic ACCEPT or REJECT decision (i.e. excluding HUMAN_REVIEW); reflects the quality of the system’s automatic decisions;
- **Auto-decision rate**: the fraction of all tasks for which an automatic decision was made; reflects the system’s productivity and its ability to relieve inspectors;
- **Human review rate**: the fraction of tasks forwarded for manual review (the complement of Auto-decision rate); characterizes the residual manual workload;
- **Critical FP rate**: the fraction of invalid tasks (those marked as bad during manual labeling) which the pipeline wrongly accepted as ACCEPT. This is the most important operational metric, since each such error means payment for work that was not performed.

95% Wilson confidence intervals are computed for each proportion to compare the two regimes statistically. The significance of the difference between specialized and unified architectures on the same set is assessed via McNemar’s test on discordant decision pairs.

6 Results

6.1 Module 2: training the Location Verification models

Within this block, four independent location-verification models were trained: three specialized (10 000 pairs each, RTX 3070) and one unified (30 000 mixed pairs, Kaggle T4). Metrics measured on the validation sets at the best checkpoint are summarized in Table 3.

Table 3. Location Verification module metrics on validation sets at the best checkpoint. “Train pos.” is the number of positive pairs in the training set after image-download losses; an equal number of *mismatched negatives* (within-category shuffle of the “after” photo across different tasks) is added, so the total training set is twice as large. Note: these mismatched negatives differ from the synthetic, augmentation-based negatives used to train Module 3 (see §4.7 and §7.2). The CP1 row is shown for comparison. The unified-model row of 24 000 corresponds to 80% of the 30 000 sampled pairs without further loss, since most pairs were already cached locally from the specialized sampling step; the specialized rows show a 0.5–3.5% reduction from the target 8 000 due to per-category image downloads.

Model	Train pos.	Val Acc	F1	AUC
CP1 baseline (ResNet18, from scratch)	~315 000	0.6910	n/a	n/a
Specialized: asphalt	7 724	0.9510	0.9501	0.9906
Specialized: lawn	7 996	0.9690	0.9691	0.9945
Specialized: curb	7 952	0.9637	0.9633	0.9921
Unified (30 000 mixed)	24 000	0.9681	0.9679	0.9942

Comparison with the CP1 baseline ($0.691 \rightarrow 0.968$) shows the combined contribution of three key changes: (a) using an ImageNet-pretrained backbone instead of training from scratch; (b) switching from ResNet18 with channel concatenation to a Siamese ResNet50 with shared weights; (c) applying standard photometric augmentations. Training curves for the unified model are shown in Fig. 3.

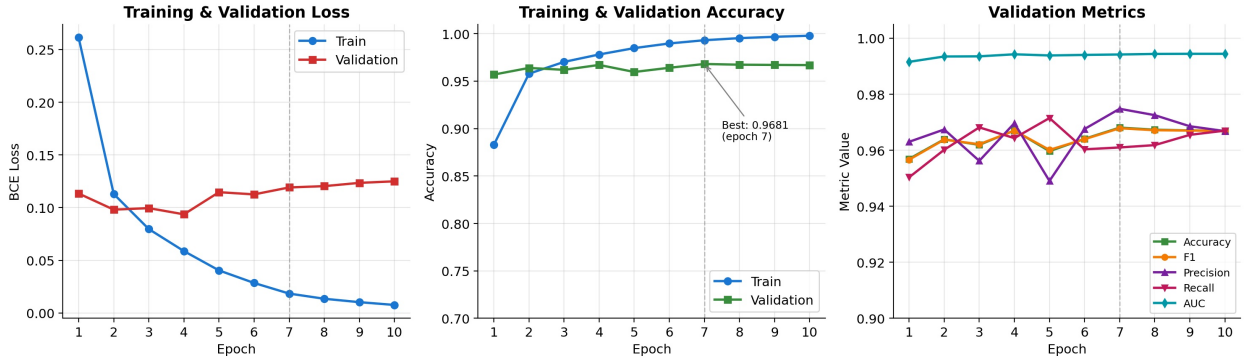


Figure 3. Training and validation curves of the unified Location Verification model: (left) BCE loss, (center) accuracy, (right) full set of validation metrics. The vertical dashed line marks the best epoch (epoch 7).

The curves show a classical picture: training loss decreases monotonically over all 10 epochs, while validation loss reaches a minimum at epoch 4 and then begins to rise, indicating the onset of overfitting. Nevertheless, validation accuracy stabilizes around 0.967 and remains there from epoch 7 through epoch 10; the early-stopping mechanism then selects the epoch-7 checkpoint as the best. Early stopping monitors validation accuracy (`val_acc`), not validation loss, so the selected checkpoint corresponds to the epoch at which accuracy last set a new maximum, rather than at which loss reached its minimum. The discrepancy between `val_loss` and `val_acc` behavior is explained in this case by the fact that after epoch 4 the model becomes less confident in its correct predictions (so BCE grows), while the decision boundary at threshold 0.5 remains stable (so accuracy does not drop).

The confusion matrix of the unified model on its validation set (12 000 pairs) is shown in Fig. 4. Almost half of all errors are false negatives (the model labels photographs of the same place as different), and this property is important for understanding the pipeline’s eventual behavior: it explains why the pipeline in unified-model mode more often sends invalid pairs to `HUMAN_REVIEW` rather than accepting them automatically (see §6.3).

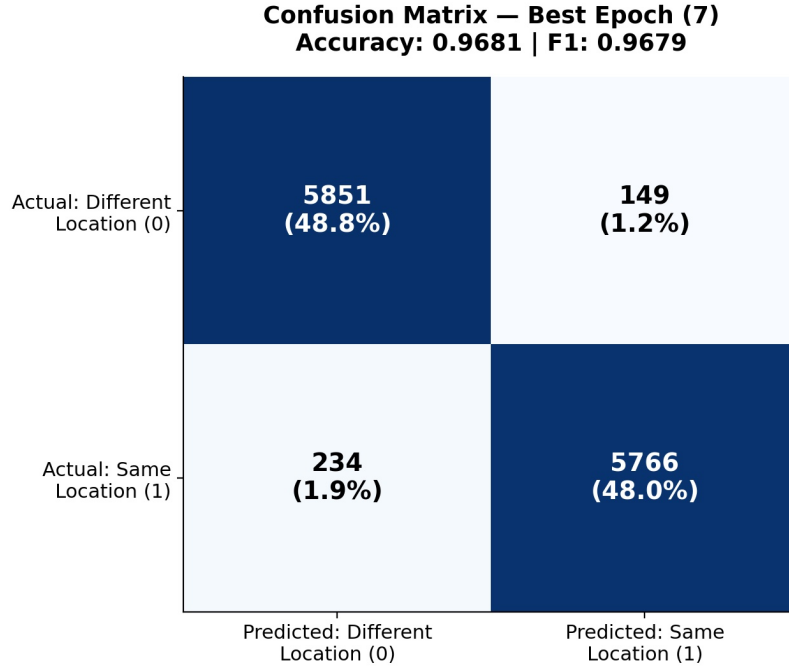


Figure 4. Confusion matrix of the unified Location Verification model on its 12 000-pair validation set (best checkpoint, epoch 7).

6.2 Module 3: training the Quality Assessment models

Three quality-assessment models were trained, one per violation category. Each model was trained on positive pairs of the corresponding category and an equal number of synthetic negative pairs generated as described in Section 4.7. All three models converge to a validation accuracy above 0.99 within 2–3 epochs.

This high a validation metric is, by itself, already a sign that the task has been trivialized: the model distinguishes positives from synthetic negatives with near-perfect confidence. Analysis of the distribution of output probabilities p_{qa} produced by the models on the full set of 6 567 UNSEEN test pairs (with the unified Module 2 in operation) confirms the suspicion. The median p_{qa} is 1.0000, the mean is 0.9929, and the interquartile range is [0.9997, 1.0000]. In only 27 cases (0.41% of the set) does the output probability fall below 0.30; that is, only in these 27 cases is Module 3 in principle capable of affecting the final decision through the REJECT rule. Of these 27 pairs, 25 correspond to valid (good) pairs and the remaining 2 to invalid (bad) pairs. In other words, in the vast majority of cases, Module 3 produces the same value, close to one, regardless of what is depicted in the photograph. This is not a consequence of a poor hyperparameter setting but reflects a fundamental property of the task in its current formulation. A detailed analysis of the causes is given in Section 7.2, and a quantitative ablation in Section 6.6.

6.3 End-to-end pipeline evaluation

The full pipeline (Modules 1-4) was run twice on the held-out UNSEEN test set (6 567 pairs): once with three specialized location-verification models, and once with the single unified model.

Crucial detail for interpreting the comparison. Both runs used the same set of three specialized quality-assessment models (Module 3); the only thing that changed between runs was Module 2 (Location Verification). Any difference in end-to-end metrics between the specialized and unified architectures is therefore explained exclusively by Module 2. This eliminates a potential confound that could arise from simultaneously replacing two components, and logically links this comparison with the ablation experiment in Section 6.6: in both cases, the contribution of Module 2 is isolated.

The results are summarized in Table 4.

Table 4. End-to-end pipeline evaluation on the held-out UNSEEN test set. The unified model wins on all three key metrics.

Metric (UNSEEN set)	Specialized (3 models)	Unified (1 model)
Total pairs evaluated	6 567	6 567
Correct ACCEPT	5 894	6 055
Correct REJECT	38	13
Wrong ACCEPT (critical FP)	18	2
Wrong REJECT	187	163
Sent to HUMAN_REVIEW	430	334
Auto-accuracy	96.66%	97.35%
Critical FP rate	19.15%	2.13%
Human review rate	6.55%	5.09%

A visual comparison of key metrics is shown in Fig. 5; the full confusion matrices are shown in Fig. 6.

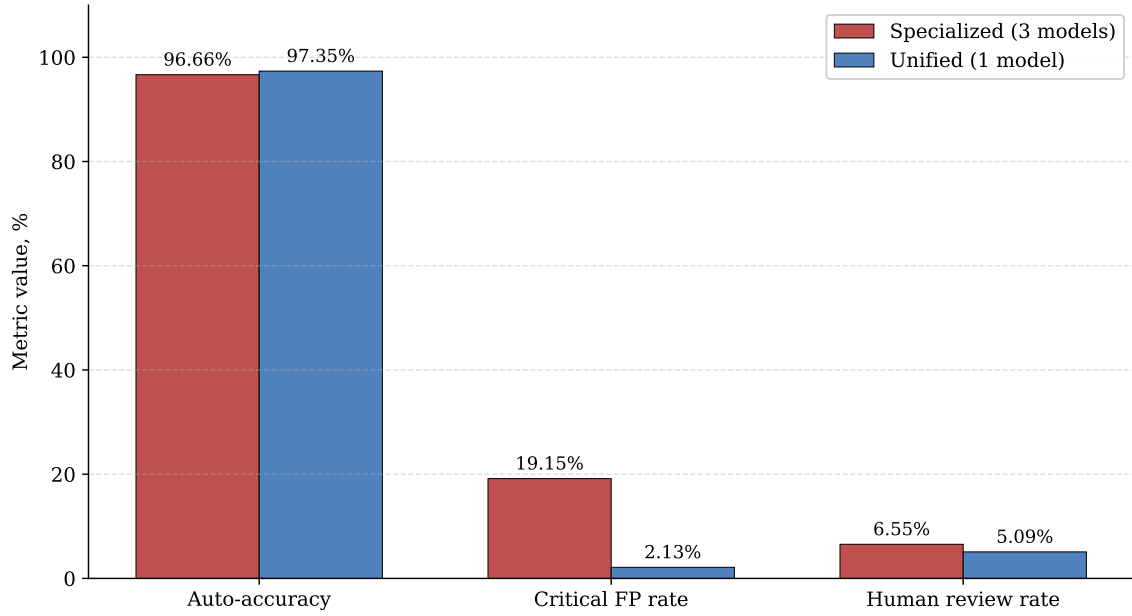


Figure 5. Comparison of key pipeline metrics for specialized vs unified modes (UNSEEN set).

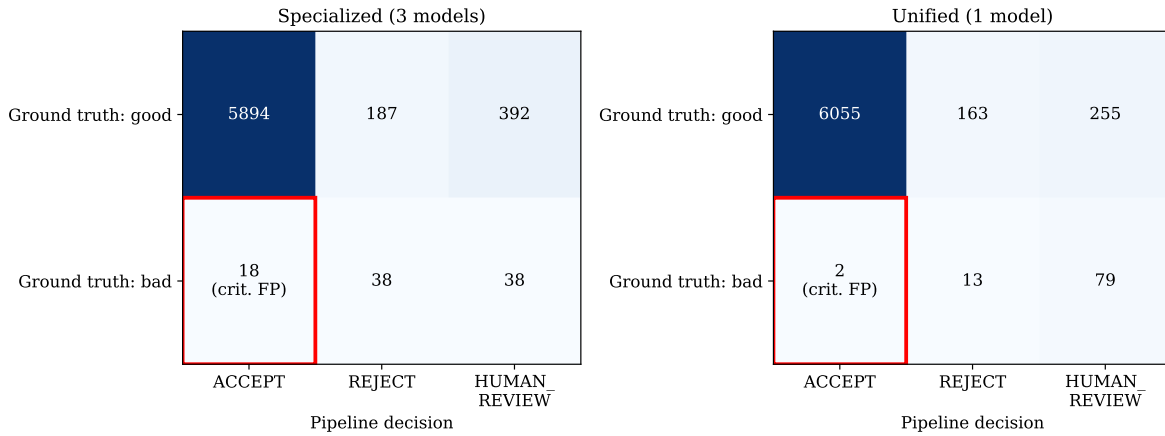


Figure 6. Full pipeline confusion matrices on UNSEEN. The wrong-ACCEPT cell (bottom-left of the second row in each panel) is highlighted with a red border and represents the operationally critical error.

The unified model dominates on both main metrics. The gain in auto-accuracy is 0.69 percentage points (from 96.66% to 97.35%), which may seem modest, but the decisive metric is the critical FP rate: the specialized ensemble lets 18 of 94 invalid pairs through as ACCEPT (19.15%), whereas the unified model lets only 2 through (2.13%), corresponding to an approximately ninefold reduction in critical error.

Since both architectures are evaluated on the same set of 94 invalid pairs, the difference should be checked with a paired McNemar test on discordant decision pairs. Of the 94 pairs, 18 were correctly rejected or sent for manual review only in the unified model, and 2 were correctly handled only in the specialized one. A two-sided exact binomial test on these 20 discor-

dant pairs gives $p \approx 0.0004$, which rejects the null hypothesis of equivalence of the two regimes by a large margin. The 95% Wilson confidence intervals for the respective false-positive rates are [12.47%, 28.25%] for the specialized architecture and [0.59%, 7.43%] for the unified architecture; the intervals do not overlap, which also confirms the significance of the difference.

The flip side of this advantage is interesting in its own right. The unified model forwards 79 of 94 invalid pairs to HUMAN_REVIEW (84%), versus only 38 (40%) for the specialized one. In other words, for the unified model, a larger fraction of invalid pairs falls into the “uncertainty zone”, whereas the specialized model more often makes a confident (but sometimes wrong) decision. In the context of the operational task at hand, this behavior of the unified model is preferable: an error in the form of a wrong ACCEPT is more costly than the additional inspector workload from HUMAN_REVIEW.

Analysis of the remaining wrong ACCEPT decisions in the unified model shows that both pairs belong to the asphalt category and have p_{loc} values just above the acceptance threshold (0.9521 and 0.9778) with $p_{\text{qa}} = 1.0000$ in both cases, meaning that Module 3 was entirely useless in detecting them. This finding aligns with the observations in §7.2.

Per-category breakdown for the unified model on UNSEEN. The distribution of errors across violation types is shown in Table 5. Both remaining critical errors are concentrated in the “asphalt” category, while in the “lawn” and “curb” categories the system did not let through a single invalid pair.

Table 5. Per-category breakdown of unified-model results on UNSEEN.

Category	n	bad	Auto-acc	Critical FP	Human review
Asphalt	2 296	46	97.02%	4.35% (2/46)	6.53%
Lawn	2 043	26	98.12%	0.00% (0/26)	3.77%
Curb stone	2 228	22	96.98%	0.00% (0/22)	4.80%

6.4 Generalization check: SEEN vs UNSEEN

Generalization quality can be checked by comparing metrics on the SEEN subset (1 410 pairs in which at least one of the two photographs was encountered by the model during training) with metrics on UNSEEN. For the unified model, auto-accuracy on SEEN is 98.16% versus 97.35% on UNSEEN, a difference of less than one percentage point. For the specialized model, the gap is noticeably larger (98.64% on SEEN vs 96.66% on UNSEEN, about two percentage points), which can also be regarded as indirect evidence of a greater tendency to overfit in the specialized architecture. Both architectures show

stable generalization from the training set to the independently labeled test set.

It is also worth noting that the shares of SEEN and UNSEEN (1 410 and 6 590 pairs respectively) can be compared to the baseline expected overlap fraction for a random sample of 30 000 pairs from 196 858 ($30000/196858 \approx 15.24\%$). The empirical SEEN share is $1410/8000 = 17.6\%$. The excess over the baseline is explained by the fact that the definition of SEEN counts at least one photograph in the pair overlapping with the training set, rather than the entire pair, and individual photographs may be reused across different pairs (for example, the same “before” photograph may appear in different tasks at the same object). The empirical values agree with the theoretical expectation in order of magnitude and confirm the correctness of the stratified-sampling strategy. The numbers above describe the total composition of each group; for metric computation only the pairs with valid predictions were used (1 410 SEEN and 6 567 UNSEEN).

6.5 Threshold sensitivity

The final pipeline decision is produced by the rule in Section 3.1 with thresholds $\tau_{\text{accept}}^{\text{loc}} = 0.95$, $\tau_{\text{accept}}^{\text{qa}} = 0.80$, $\tau_{\text{reject}}^{\text{loc}} = \tau_{\text{reject}}^{\text{qa}} = 0.30$. To assess sensitivity of the results to these values, metrics on the UNSEEN set with the unified Module 2 were measured for varying $\tau_{\text{accept}}^{\text{loc}}$ with the other thresholds fixed; the results are summarized in Table 6.

Table 6. Effect of the $\tau_{\text{accept}}^{\text{loc}}$ threshold on pipeline metrics (unified model, UNSEEN). Other thresholds are fixed at $\tau_{\text{accept}}^{\text{qa}} = 0.80$, $\tau_{\text{reject}}^{\text{loc}} = \tau_{\text{reject}}^{\text{qa}} = 0.30$.

$\tau_{\text{accept}}^{\text{loc}}$	Auto-accuracy	Critical FP rate	Auto-rate	Wrong ACCEPT
0.80	97.10%	23.40%	97.20%	22
0.85	97.20%	15.96%	96.71%	15
0.90	97.29%	8.51%	96.07%	8
0.93	97.32%	5.32%	95.48%	5
0.95	97.35%	2.13%	94.91%	2
0.97	97.34%	1.06%	93.94%	1
0.99	97.29%	0.00%	91.66%	0

The value $\tau_{\text{accept}}^{\text{loc}} = 0.95$ corresponds to a reasonable trade-off: the critical FP rate is already an order of magnitude lower than at 0.80, while the loss in auto-decision rate (3 p.p.) remains moderate. Raising the threshold further to 0.99 eliminates wrong ACCEPT decisions entirely, but at the cost of increasing inspector workload from 5.1% to 8.3%. The choice of the spe-

cific threshold is an operational decision for the customer and can be adjusted without retraining the models.

An analogous analysis of the $\tau_{\text{accept}}^{\text{qa}}$ threshold showed that its value in the range 0.5–0.99 has practically no effect on either the critical FP rate (remains 2.13% in all cases) or on auto-accuracy (varies within 0.1 p.p.). This is consistent with the observation in §6.2 of the degenerate distribution of p_{qa} , and naturally leads to the next ablation.

6.6 Ablation: pipeline without Module 3

Given the §6.2 observation about the suspicious concentration of p_{qa} near unity, the next ablation removes Module 3 from the pipeline altogether. The removal was performed without retraining the remaining components: for each UNSEEN pair we used the already-computed p_{loc} scores of the unified and specialized models, and reformulated the decision rule so that it depends only on p_{loc} :

- if $p_{\text{loc}} < 0.30$, then REJECT;
- if $p_{\text{loc}} > 0.95$, then ACCEPT;
- otherwise HUMAN_REVIEW.

Results for the full pipeline versus the simplified pipeline are shown in Table 7.

Table 7. Ablation: full pipeline vs simplified (without Module 3). UNSEEN, 6 567 pairs, 94 invalid.

Metric	Unified		Specialized	
	Full	No M3	Full	No M3
Correct ACCEPT	6 055	6 096	5 894	5 933
Correct REJECT	13	12	38	37
Wrong ACCEPT (critical FP)	2	2	18	18
Wrong REJECT	163	141	187	165
Sent to HUMAN_REVIEW	334	316	430	414
Auto-accuracy	97.35%	97.71%	96.66%	97.00%
Critical FP rate	2.13%	2.13%	19.15%	19.15%
Human review rate	5.09%	4.81%	6.55%	6.30%

The result is unambiguous: removing Module 3 not only does not degrade any of the key metrics but produces a small, systematic improvement in auto-accuracy (+0.36 p.p. for the unified architecture) and reduces inspector workload (−0.28 p.p.). The critical FP rate does not change in either regime, confirming that none of the critical errors had ever been caught by Module 3.

This ablation result shows that the current implementation of the Quality Assessment module is redundant and should either be removed from the production pipeline or reworked. A detailed discussion of the causes and possible directions for reworking is given in Section 7.2.

7 Discussion

7.1 Why the unified model outperformed the specialized one

The initial hypothesis was that category-specialized models would outperform a single unified model: each specialized model should learn the visual patterns of only one violation type and form more saturated category-specific features. The experimental result refutes this on both main metrics.

The most likely explanation is the relative volume of the training sets. Specialized models were trained on roughly 8 000 positive pairs each, whereas the unified model used 30 000 positive pairs from the mixed corpus. A four-fold increase in training-set size apparently outweighs the benefit of narrow model specialization, at least at the model capacity used here (26.7 million parameters).

A second factor is that the unified model encounters a wider range of variation in irrelevant factors (different scenes, different cameras, different seasons), which forces it to form more abstract features and ignore visual noise. The specialized models may overfit to the narrow visual distribution of their category and handle non-standard cases worse. A telling example is invalid pairs in which a PDF document or GIS-system screenshot is submitted in place of an urban-environment photograph: the unified model, having seen more diverse “unrelated” images, more confidently recognizes such pairs as different locations.

The unified model’s advantage is most pronounced on the operationally critical metric: the rate of wrong ACCEPTs. When the unified model is uncertain about an invalid pair, it routes the pair to HUMAN_REVIEW rather than making an automatic decision. This is a safe failure mode. For the task at hand, it is preferable to the risk of missing work that was not performed.

7.2 Limitations of the synthetic-negatives approach

All three quality-assessment models converged to a validation accuracy above 0.99 within 2–3 epochs. This is a suspiciously high value and indicates that, in its current formulation, the task is trivial for the model. Visual analysis

of augmented pairs in the `visualize_augmentation.py` script confirms the suspicion: even with the `SafeGeometricAug` class that prevents obvious black border voids, the cumulative artifacts of the augmentation pipeline (crop borders, banding from gamma compression, spectral signature of Gaussian blur) prove sufficient for the model to distinguish synthetic from real photographs without engaging with semantic content.

In other words, the trained model is most likely solving the task “is this an augmented image?” rather than “has the violation actually been repaired?”. In the full pipeline this means Module 3 mainly contributes noise; the high end-to-end quality is achieved chiefly through Module 2 (Location Verification). Table 4 supports this: prevention of wrong ACCEPTs is mostly due to Module 2 identifying bad pairs as “different locations” ($p_{\text{loc}} < 0.30$), not Module 3 detecting “violation not repaired” ($p_{\text{qa}} < 0.30$).

This limitation is methodological and, in all likelihood, cannot be removed within the framework of synthetic negatives generated from the same source data as the positives. A more rigorous approach would require one of the following data sources: (a) real rejected tasks from inspectors (absent from the data studied); (b) a purpose-built set of intentionally inconsistent pairs of the same location, “before”-and-“after”-rephotographed without performing the repair; (c) a reformulation of the task such that the distribution of augmentations coincides with the actual distribution of “rephotographed, unrepaired” photographs, which is itself poorly defined.

The ablation experiment described in Section 6.6 formally confirms the claim above: removing Module 3 from the pipeline does not degrade any key metric and gives a small improvement in auto-accuracy. In its current implementation Module 3 is not merely useless but mildly harmful. This makes it a natural recommendation to remove it from the production version until real labeled negative examples become available.

Despite the limitations noted, it seems honest to keep the description of the approach and its results within the work as an example of a methodologically interesting negative result, and as a starting point for follow-up work.

7.3 Other limitations

Several other methodological limitations of the work should be made explicit, beyond the quality-assessment module discussed above.

Single annotator. The reference labeling of 8 000 pairs was performed by a single person (the author). For the bad category with its low frequency (1.4% of all pairs) this is especially important: the determination “violation clearly not repaired” may rely on subjective criteria in ambiguous cases. A rigorous quality evaluation would ideally involve a second annotator and a

measure of agreement (e.g. Cohen’s κ), which was not feasible within the scope of this work.

Training seeds. Although pair sampling and stratified train/val splitting use a fixed seed of 42 (see Section 4.4), the neural-network training procedure does not explicitly fix the seed for random head initialization or DataLoader batch order. This means that re-running training may yield slightly different model weights and metrics; the magnitude of such fluctuation has not been measured.

Geographic and seasonal narrowness of the data. All data were collected in Moscow; application to other cities or regions would require at least fine-tuning, and possibly re-labeling for new types of urban environments. The exact time interval of data collection is also not stated in the work, and seasonal effects (winter snow cover, foliage coloration, lighting) on the model have not been studied.

Invalid-pair class. The labeled set contains only 94 invalid pairs against 6 567 valid predictions on UNSEEN. This gives satisfactory but not ideal statistical power for resolving small differences in the critical FP rate. The Wilson confidence intervals (see §6.3) show that, for small proportions, the spread can reach several percentage points even with zero observed errors. The statement “critical FP rate is close to zero” in this work should therefore be understood as “no higher than a few percent with 95% confidence”.

Lack of a training-set size ablation. The hypothesis explaining the superiority of the unified model by the larger training-set volume (§7.1) has not been formally tested by an ablation in which the unified model would be trained on a volume comparable to the specialized ones. This experiment is listed in future research directions (§8).

7.4 Operational implications

In unified-model mode, the system handles about 95% of incoming tasks automatically with 97.35% accuracy. Inspectors are freed from routine processing of most cases and can focus on the remaining $\sim 5\%$ borderline ones. A critical FP rate of 2.13% means that approximately 2 of every 100 invalid tasks would be wrongly accepted automatically. This is substantially better than the specialized ensemble (19 of 100) and is, apparently, acceptable for an MVP, although further reduction of this figure is desirable.

In terms of manual-review costs, the 5.09% human-review rate represents a moderate and predictable workload: at a flow of 196 858 tasks per year, this corresponds to approximately 10 000 tasks requiring manual handling, more than 20 times below the current load. The extra inspector

throughput can be directed at more complex cases and at improving labeling quality for future model updates.

8 Conclusion

This work shows that verifying urban-infrastructure repair from before/after photograph pairs can be largely automated through an end-to-end Siamese neural network pipeline. The pipeline accepts an Excel file with tasks in the format used by city services, passes each task through four sequential modules (category router, location verification, quality assessment, decision router), and produces an output Excel file with one of three possible decisions: ACCEPT, REJECT, or HUMAN_REVIEW.

The principal experiment compared two architectural approaches for the central location-verification module: an ensemble of three specialized models (one per category) versus a single unified model trained on the mixed corpus. The comparison was carried out on a held-out labeled sample of 6 567 pairs not seen by any model during training. Contrary to the initial hypothesis, the unified model outperformed the specialized ensemble on both key metrics: auto-accuracy was 97.35% vs 96.66%, and the critical false-positive rate was 2.13% vs 19.15%. The operationally most important metric, the rate of wrong automatic acceptance of invalid tasks, was roughly nine times lower in the unified-model regime. The difference is confirmed by a paired McNemar test ($p \approx 0.0004$) and by non-overlapping 95% Wilson confidence intervals. This result is, in all likelihood, related to the four-fold larger training set of the unified model relative to any specialized one; however, a formal causal claim is not established in this work, since it would require an additional ablation by training-set size, which is included in future work directions (see below).

The second principal decision was to train the quality-assessment module using synthetic negatives formed by strong augmentation of the source photographs. This approach yields a suspiciously high validation accuracy (above 0.99 by epoch 3). The task has been trivialized. The distribution of Module 3 output probabilities on the test set turned out to be strongly concentrated near unity (median 1.0000, mean 0.9929), and an ablation experiment showed that removing Module 3 from the pipeline does not degrade any key metric and even slightly improves auto-accuracy (from 97.35% to 97.71%). This result is honestly reported as a methodological limitation of the work; it motivates excluding Module 3 from the current production version and pursuing future work either to collect real rejected pairs or to reformulate the task.

The final product is implemented as the command-line utility `mvp_`

pipeline.py, which consumes Excel files in the city services' current format and produces an Excel file containing per-task results and accompanying confidence scores. The utility is ready for integration into the existing inspection-service workflow. All source code, trained models, and reference labels are released in an open GitHub repository accompanying this work.

Future research directions: an ablation by training-set size (training the unified model on 10 000 pairs to directly test the §7.1 hypothesis); expanding the list of supported violation categories; integrating an additional modality (the textual task description) as an auxiliary input to the model; developing a visual-explanation module (Grad-CAM or similar) for HUMAN_REVIEW cases; and collecting a real labeled set of rejected tasks for proper training and evaluation of the quality-assessment module, or reworking that module entirely. Two operational tasks remain: measuring pipeline throughput on the target infrastructure and engaging a second annotator to assess inter-annotator agreement on the bad class.

References

- [1] Christoph Koch, Kamal Georgieva, Varun Kasireddy, Burcu Akinci, Paul Fieguth. A review on computer vision based defect detection and condition assessment of concrete and asphalt civil infrastructure // *Advanced Engineering Informatics*. 2015. Vol. 29, No. 2. P. 196–210.
- [2] Henrique Oliveira, Paulo Lobato Correia. Automatic road crack detection and characterization // *IEEE Transactions on Intelligent Transportation Systems*. 2013. Vol. 14, No. 1. P. 155–168.
- [3] Kasthurirangan Gopalakrishnan, Siddhartha K. Khaitan, Alok Choudhary, Ankit Agrawal. Deep Convolutional Neural Networks with transfer learning for computer vision-based data-driven pavement distress detection // *Construction and Building Materials*. 2017. Vol. 157. P. 322–330.
- [4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Deep Residual Learning for Image Recognition // *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016. P. 770–778.
- [5] Mingxing Tan, Quoc V. Le. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks // *Proceedings of the International Conference on Machine Learning (ICML)*. 2019. P. 6105–6114.
- [6] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, Roopak Shah. Signature verification using a Siamese time delay neural network // *Advances in Neural Information Processing Systems (NeurIPS)*. 1994. P. 737–744.
- [7] Florian Schroff, Dmitry Kalenichenko, James Philbin. FaceNet: A Uni-

- fied Embedding for Face Recognition and Clustering // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015. P. 815–823.
- [8] Christoph Zauner. Implementation and benchmarking of perceptual image hash functions. Master’s thesis. Upper Austria University of Applied Sciences, 2010.
- [9] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, Eero P. Simoncelli. Image quality assessment: from error visibility to structural similarity // IEEE Transactions on Image Processing. 2004. Vol. 13, No. 4. P. 600–612.
- [10] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, Oliver Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric // Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2018. P. 586–595.
- [11] Nils Reimers, Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks // Proceedings of EMNLP-IJCNLP. 2019. P. 3982–3992.
- [12] Alec Radford, Jong Wook Kim, Chris Hallacy et al. Learning Transferable Visual Models From Natural Language Supervision // Proceedings of the International Conference on Machine Learning (ICML). 2021. P. 8748–8763.

Appendix A. Project repository structure

The open GitHub repository accompanying this work contains the following files, organized into functional groups.

A.1. Data preparation and cleaning

- `filter_pairs.py`: Tkinter-based GUI for manual pair labeling (produces `pair_decisions.csv`);
- `clean_dataset.py`: automatic URL filtering by extension (drops non-image files);
- `fix_bad_decisions.py`: automatic detection and correction of labeling conflicts (see Stage 4 in §4.3);
- `detect_anomalies.py`: CLIP-based detection of suspicious pairs (PDF documents, screenshots, empty images);
- `review_suspicious.py`: a second GUI pass for re-checking flagged pairs;
- `download_rejected.py`: a utility for downloading images from manually rejected pairs;
- `prepare_training_data.py`: conversion of XLSX into train/val CSV with

- synthetic negatives and parallel image downloads;
- `prepare_quality_data.py`: conversion of existing train/val CSV into the format used to train Quality Assessment;
- `visualize_augmentation.py`: a diagnostic utility for visual inspection of augmentation outputs.

A.2. Model training

- `train_siamese.py`: training of location-verification models (specialized and unified);
- `train_quality.py`: training of quality-assessment models, including the `SafeGeometricAug` class;
- `plot_results.py`: plotting of training curves and confusion matrices from training logs.

A.3. Pipeline and evaluation

- `mvp_pipeline.py`: the main CLI utility (input XLSX, output XLSX with ACCEPT/REJECT/HUMAN_REVIEW decisions);
- `evaluate_pipeline.py`: end-to-end evaluation script comparing the specialized and unified regimes on `pair_decisions.csv` with overlap detection against the training set (SEEN vs UNSEEN);
- `evaluate_on_val.py`: an alternative evaluation on held-out validation sets (without the need to re-download images from the internet).

A.4. Data and trained models

- `pair_decisions.csv`: manual labeling of 8 000 pairs (7 888 good + 112 bad);
- `train_asphalt.csv`, `val_asphalt.csv`, `train_lawn.csv`, `val_lawn.csv`, `train_curb.csv`, `val_curb.csv`: training and validation sets for location verification;
- `train_qa_*.csv`, `val_qa_*.csv`: training and validation sets for quality assessment;
- `runs/asphalt/best_model.pt`, `runs/lawn/best_model.pt`, `runs/curb/best_model.pt`: weights of the specialized Location Verification models;
- `runs/siamese_full/best_model.pt`: weights of the unified Location Verification model;
- `runs/qa_asphalt/best_model.pt`, `runs/qa_lawn/best_model.pt`, `runs/qa_curb/best_model.pt`: weights of the Quality Assessment models;

- `evaluation_summary.txt`, `evaluation_results.csv`: final pipeline-evaluation output files, which form the basis of Section 6 of this work.

A.5. Documentation

- `README.md`: a repository overview, installation instructions, and usage examples.